

```
import numpy as np
import pandas as pd
import os
from sklearn.ensemble import AdaBoostRegressor
from sklearn.tree import DecisionTreeRegressor

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, StackingRegressor
from sklearn.linear_model import Ridge
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import ConfusionMatrixDisplay
# Visualisation libraries
import matplotlib.pyplot as plt
# %matplotlib inline
import seaborn as sns

import plotly.express as px
import plotly.graph_objects as go
```

```
dfi=pd.read_csv('air_quality_with_aqi.csv')

# # Extract delhi's data

# df=city_day_data.groupby('StationId').get_group('DL001')

dfi.head()
```



	Timestamp	PM2.5 (µg/m³)	PM10 (µg/m³)	NO (µg/m³)	NO2 (µg/m³)	NOx (ppb)	NH3 (µg/m³)	SO2 (µg/m³)	CO (mg/m³)	Ozone (µg/m³)	...	SR (W/mt2)	BP (mmHg)	VWS (m/s)	AQI_PM2.5	AQI_
0	2024-01-01 00:00:00	190.00	266.00	17.72	61.20	47.03	29.50	25.85	1.35	15.28	...	5.38	991.90	1.02	353.953488	2
1	2024-01-01 01:00:00	191.00	283.00	14.15	62.95	45.03	34.02	27.30	1.27	19.40	...	5.32	991.92	1.00	354.720930	2
2	2024-01-01 02:00:00	183.75	278.50	8.10	67.80	42.65	25.95	26.90	1.20	16.50	...	5.28	991.92	0.92	349.156977	2
3	2024-01-01 03:00:00	182.50	272.75	3.92	66.45	38.55	20.00	24.25	1.23	15.63	...	5.15	991.90	0.95	348.197674	2
4	2024-01-01 04:00:00	193.75	279.75	7.00	65.68	40.60	23.25	26.08	1.35	11.75	...	5.08	991.90	0.90	356.831395	2

5 rows × 32 columns

```
df=dfi.drop(['Benzene (µg/m³)', 'Toluene (µg/m³)', 'Xylene (µg/m³)', 'O Xylene (µg/m³)', 'Eth-Benzene (µg/m³)', 'MP-Xylene (µg/m³)', 'TOT-RF (mm
df.head()
```



	Timestamp	PM2.5 (µg/m³)	PM10 (µg/m³)	NO (µg/m³)	NO2 (µg/m³)	NOx (ppb)	NH3 (µg/m³)	SO2 (µg/m³)	CO (mg/m³)	Ozone (µg/m³)	AT (°C)	RH (%)	WS (m/s)	WD (deg)	RF (mm)	SR (w/mt2)	
0	2024-01-01 00:00:00	190.00	266.00	17.72	61.20	47.03	29.50	25.85	1.35	15.28	10.50	75.22	0.32	177.00	NaN	5.38	95
1	2024-01-01 01:00:00	191.00	283.00	14.15	62.95	45.03	34.02	27.30	1.27	19.40	10.30	75.68	0.38	191.50	NaN	5.32	95
2	2024-01-01 02:00:00	183.75	278.50	8.10	67.80	42.65	25.95	26.90	1.20	16.50	10.22	75.75	0.45	191.25	NaN	5.28	95
3	2024-01-01	182.50	272.75	3.92	66.45	38.55	20.00	24.25	1.23	15.63	9.95	75.43	0.40	200.75	NaN	5.15	95

```
# Convert 'Datetime' column to datetime objects using the specified format "24-11-2017 17:00"
# df['Time'] = pd.to_datetime(df['Timestamp'], format='%d-%m-%Y %H:%M')
df['Time'] = pd.to_datetime(df['Timestamp'], format='%Y-%m-%d %H:%M:%S')

# Convert Datetime to numerical value (seconds since epoch)
df['Datetime_num'] = df['Time'].astype(np.int64) // 10**9

# Shift the AQI column 72 hours ahead (since the data is hourly, shift by 72 rows)
df['target'] = df['AQI'].shift(-72)

df=df.drop(['Timestamp'],axis=1)
df=df.drop(['Time'],axis=1)

df=df.drop(['Datetime_num'],axis=1)
df=df.drop(['RF (mm)'],axis=1)
df=df.dropna()
df['PM2.5_to_PM10_ratio'] = df['PM2.5 (µg/m³)'] / df['PM10 (µg/m³)']
df.head()
```



	PM2.5 (µg/m³)	PM10 (µg/m³)	NO (µg/m³)	NO2 (µg/m³)	NOx (ppb)	NH3 (µg/m³)	SO2 (µg/m³)	CO (mg/m³)	Ozone (µg/m³)	AT (°C)	RH (%)	WS (m/s)	WD (deg)	SR (w/mt2)	BP (mmHg)	VWS (m/s)	
0	190.00	266.00	17.72	61.20	47.03	29.50	25.85	1.35	15.28	10.50	75.22	0.32	177.00	5.38	991.90	1.02	353.5
1	191.00	283.00	14.15	62.95	45.03	34.02	27.30	1.27	19.40	10.30	75.68	0.38	191.50	5.32	991.92	1.00	354.7
2	183.75	278.50	8.10	67.80	42.65	25.95	26.90	1.20	16.50	10.22	75.75	0.45	191.25	5.28	991.92	0.92	349.7
3	182.50	272.75	3.92	66.45	38.55	20.00	24.25	1.23	15.63	9.95	75.43	0.40	200.75	5.15	991.90	0.95	348.7
4	193.75	279.75	7.00	65.68	40.60	23.25	26.08	1.35	11.75	10.02	75.62	0.30	186.00	5.08	991.90	0.90	356.8

```
df.isnull().sum()
```



	0
PM2.5 (µg/m³)	0
PM10 (µg/m³)	0
NO (µg/m³)	0
NO2 (µg/m³)	0
NOx (ppb)	0
NH3 (µg/m³)	0
SO2 (µg/m³)	0
CO (mg/m³)	0
Ozone (µg/m³)	0
AT (°C)	0
RH (%)	0
WS (m/s)	0
WD (deg)	0
SR (W/mt2)	0
BP (mmHg)	0
VWS (m/s)	0
AQI	0
target	0
PM2.5_to_PM10_ratio	0

dtype: int64

```
# Create AQI category column
def categorize_aqi(aqi):
    if aqi <= 50:
        return 'good'
    elif aqi <= 100:
        return 'satisfactory'
    elif aqi <= 200:
        return 'moderate'
    elif aqi <= 300:
        return 'poor'
    elif aqi <= 400:
        return 'verypoor'
    else:
        return 'severe'

df['AQI_category'] = df['AQI'].apply(categorize_aqi)
df=df.drop(['AQI'],axis=1)
# df['AQI_category'] = df['AQI_PM2.5'].apply(categorize_aqi)
# df = df.drop(['AQI_PM2.5'], axis=1)
df.head()
```



	PM2.5 (µg/m³)	PM10 (µg/m³)	NO (µg/m³)	NO2 (µg/m³)	NOx (ppb)	NH3 (µg/m³)	SO2 (µg/m³)	CO (mg/m³)	Ozone (µg/m³)	AT (°C)	RH (%)	WS (m/s)	WD (deg)	SR (W/mt2)	BP (mmHg)	VWS (m/s)	
0	190.00	266.00	17.72	61.20	47.03	29.50	25.85	1.35	15.28	10.50	75.22	0.32	177.00	5.38	991.90	1.02	372.5
1	191.00	283.00	14.15	62.95	45.03	34.02	27.30	1.27	19.40	10.30	75.68	0.38	191.50	5.32	991.92	1.00	369.8
2	183.75	278.50	8.10	67.80	42.65	25.95	26.90	1.20	16.50	10.22	75.75	0.45	191.25	5.28	991.92	0.92	369.6
3	182.50	272.75	3.92	66.45	38.55	20.00	24.25	1.23	15.63	9.95	75.43	0.40	200.75	5.15	991.90	0.95	370.6
4	193.75	279.75	7.00	65.68	40.60	23.25	26.08	1.35	11.75	10.02	75.62	0.30	186.00	5.08	991.90	0.90	366.2

```
X=df.drop(['target','AQI_category'],axis=1)
y=df['AQI_category']
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# plt.figure(figsize=(10,8))
# sns.heatmap(X_train.corr(), annot=True, cmap='coolwarm')
# plt.show()
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# # Build the Random Forest model
# model = RandomForestRegressor(n_estimators=100, random_state=42)
# model.fit(X_train, y_train)

# # Predict on the test set and evaluate the model
# y_pred = model.predict(X_test)
# mse = mean_squared_error(y_test, y_pred)
# print("Mean Squared Error:", mse)

# r2 = r2_score(y_test, y_pred)
# print("R^2 Score:", r2)

# estimators = [
#     ('rf', RandomForestRegressor(n_estimators=100, random_state=42)),
#     ('gbr', GradientBoostingRegressor(n_estimators=100, random_state=42))
# ]

# # Define stacking regressor with a meta model (here, Ridge regression)
# stacking_reg = StackingRegressor(
#     estimators=estimators,
#     final_estimator=Ridge()
# )

# # Train the stacking model
# stacking_reg.fit(X_train, y_train)

# # Evaluate the model
# y_pred = stacking_reg.predict(X_test)
# mse = mean_squared_error(y_test, y_pred)
# r2 = r2_score(y_test, y_pred)

# print("Stacking Model MSE:", mse)
# print("Stacking Model R^2:", r2)

clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# Make predictions
y_pred = clf.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

```

➡ Accuracy: 0.9556692242114238

Confusion Matrix:

```

[[ 63  0  0  0  0  0]
 [ 3 413  1 13  0  0]
 [ 0  8 151  0  0  0]
 [ 3  2  0 193  0  0]
 [ 0  0  0  0 89  0]
 [ 0 12  8  2  0 212]]

```

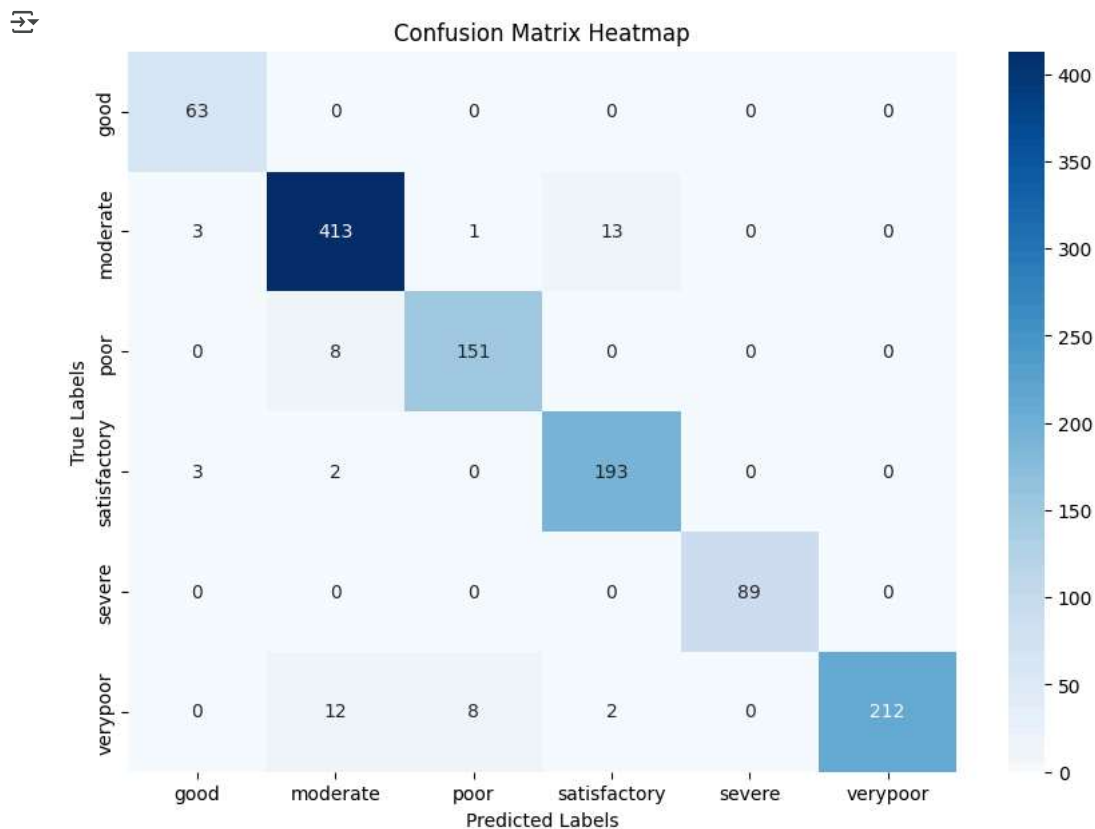
Classification Report:

	precision	recall	f1-score	support
good	0.91	1.00	0.95	63
moderate	0.95	0.96	0.95	430
poor	0.94	0.95	0.95	159
satisfactory	0.93	0.97	0.95	198
severe	1.00	1.00	1.00	89
verypoor	1.00	0.91	0.95	234
accuracy			0.96	1173
macro avg	0.96	0.97	0.96	1173

weighted avg 0.96 0.96 0.96 1173

```
# from google.colab import drive
# drive.mount('/content/drive')
```

```
# Plot confusion matrix
plt.figure(figsize=(10, 7))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=clf.classes_, yticklabels=clf.classes_)
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix Heatmap')
plt.show()
```



```
report = classification_report(y_test, y_pred, output_dict=True)
df_report = pd.DataFrame(report).transpose()
```

```
# Plot classification report
plt.figure(figsize=(10, 5))
sns.heatmap(df_report.iloc[:-1, :-1], annot=True, cmap='coolwarm', cbar=False)
plt.title('Classification Report Heatmap')
plt.show()
```

